



AIQUAA · AUTOMATIZACIÓN DE PRUEBAS DE SOFTWARE

Patrón Pre-request + Post-request con validación SQL en Postman

Guía paso a paso para entender y reutilizar el patrón de automatización end-to-end de la carpeta `E2E - Flujos con validación SQL`: cómo se declara una variable/función global una sola vez, cómo se hace dinámico solo el campo del body que necesita variar entre casos, y cómo se valida antes y después contra la base de datos.

Profesor: Steven Gilbert Gracia Ayala

Curso: Automatización de Pruebas de Software

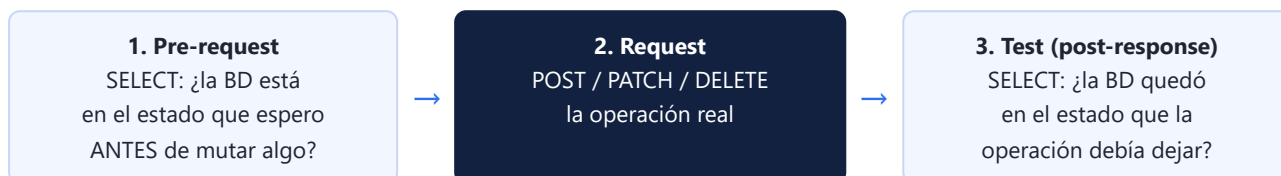
Colección: `postman_collection.json` · API: aiqaaa Sandbox API

Basado en el patrón usado en colecciones reales de automatización bancaria (SPI/CORE).

1 ¿Qué problema resuelve este patrón?

Un test de API "normal" solo revisa la respuesta HTTP: código de estado y forma del JSON. Eso no prueba que la operación realmente haya hecho lo que dice hacer en la base de datos. Un endpoint podría devolver **201 Created** con un JSON perfecto y, por un bug, no haber insertado nada — o haber insertado el dato con el estado equivocado.

El patrón agrega una segunda verificación, **directa contra la base**, en dos momentos:



Por qué importa: si el paso 3 solo mirara la respuesta del paso 2, un bug que devuelve un 201 falso positivo (sin insertar nada) pasaría desapercibido. El SELECT final es la prueba real.

2 Vocabulario de Postman que usa el patrón

Concepto	Qué es	Dónde se edita en Postman
Pre-request Script	Código JS que corre <i>antes</i> de mandar el request.	Puede estar en la colección (corre antes de TODOS los requests) o en un request individual (pestaña "Pre-request Script" de ese item).
Tests Script	Código JS que corre <i>después</i> de recibir la respuesta.	Pestaña "Tests" (o "Scripts > Post-response") de cada request.
Collection Variable	Variable compartida por toda la colección, visible desde cualquier request/script.	<code>pm.collectionVariables.get/set(...)</code>
pm.variables	Variable "local", vive solo durante la ejecución de un mismo request (su pre-request y su test comparten este scope).	<code>pm.variables.get/set(...)</code>
pm.sendRequest	Manda un HTTP request adicional desde dentro de un script, sin que aparezca como un item más de la colección.	Se usa dentro de pre-request/test para consultar la base vía el endpoint SQL.

3 Paso a paso: qué declara cada componente

3.1 — Pre-request Script de la COLECCIÓN (se declara una sola vez)

Se edita haciendo clic en el nombre de la colección (no en un request) → pestaña **Scripts** → **Pre-request script**. Corre antes de *cada* request de la colección, así que es el lugar correcto para declarar cosas que

todos van a reutilizar — incluyendo los **valores por defecto** del body de "Crear transferencia":

```
if (typeof utils === "undefined") {
  utils = {
    bodySqlRest: function (sql, params) {
      return {
        url: pm.collectionVariables.get("baseUrl") + "/api/v1/sql/select",
        method: "POST",
        header: {
          "Content-Type": "application/json",
          "x-api-key": pm.collectionVariables.get("apiKey")
        },
        body: { mode: "raw", raw: JSON.stringify({ sql: sql, params: params || [] }) }
      };
    }
  };
}

// Valores por defecto del body de "Crear transferencia". Se resetean a su
// default ANTES DE CADA request (porque este script corre siempre primero).
pm.collectionVariables.set("cuentaOrigenId", 1);
pm.collectionVariables.set("cuentaDestinoId", 2);
pm.collectionVariables.set("monto", 25000);
pm.collectionVariables.set("descripcion", "Pago de alquiler - E2E");
```

- A** `utils.bodySqlRest(sql, params)` es una *función*, no una variable de Postman: vive en memoria durante toda la corrida de la colección. Su trabajo es armar el objeto que después se le pasa a `pm.sendRequest(...)` para pegarle a `/api/v1/sql/select` — la misma URL, el mismo header `x-api-key` y el mismo formato de body (`{ sql, params }`) en un solo lugar. Sin esto, cada pre-request/test tendría que repetir esas 10 líneas.
- B** `cuentaOrigenId`, `cuentaDestinoId`, `monto` y `descripcion` sí son *collection variables* (persisten y son visibles desde cualquier request) — **una por cada campo del body que en algún caso va a variar**. No hay un objeto "template" que clonar: cada campo es su propia variable, con su propio valor por defecto.

Ojo: si `utils` no está declarado — por ejemplo, corriste un solo request suelto en vez de la colección completa desde el Runner — el script va a tirar `ReferenceError: utils is not defined`. Por eso conviene correr la carpeta completa, no un request aislado, la primera vez.

3.2 — El Body del request: solo el campo que cambia es `{{variable}}`

Clave del patrón: el body queda como un JSON normal y legible — no hace falta convertir todo el body en una sola variable. Cada campo usa su propia variable donde puede necesitar cambiar, y el resto queda igual que un body fijo cualquiera:

```
{
  "cuentaOrigenId": {{cuentaOrigenId}},
  "cuentaDestinoId": {{cuentaDestinoId}},
  "monto": {{monto}},
  "descripcion": "{{descripcion}}"
}
```

- A Postman reemplaza cada `{{variable}}` por su valor actual, como texto, justo antes de mandar el request. Para un número (`monto` , `cuentaOrigenId`) el placeholder va **sin comillas** — así el JSON final queda `"monto": 25000` y no `"monto": "25000"` . Para un string (`descripcion`) el placeholder va **entre comillas**, como cualquier valor de texto en JSON.
- B El orden importa: Postman siempre corre el Pre-request Script completo (el de la colección y después el del request) antes de armar/enviar el request — por eso, para cuando Postman lee este body, las variables ya tienen el valor que les corresponde a este caso puntual.

3.3 — Pre-request Script del REQUEST: sobrescribir solo lo que hace falta

Esto es la pestaña "Pre-request Script" de ese request puntual. El caso feliz ("Crear transferencia") ni siquiera necesita tocar el body — usa los defaults que puso el pre-request de la colección — así que solo valida la precondition en la BD:

```
// No hace falta sobrescribir ninguna variable: el caso feliz usa los
// defaults que puso el Pre-request Script de la COLECCION.

// Valida la precondition: que las cuentas existan y esten activas
// ANTES de mandar el POST.
const cuentaOrigenId = Number(pm.collectionVariables.get("cuentaOrigenId"));
const cuentaDestinoId = Number(pm.collectionVariables.get("cuentaDestinoId"));
const request = utils.bodySqlRest(
  "SELECT id, activa FROM cuentas WHERE id IN ($1, $2)",
  [cuentaOrigenId, cuentaDestinoId]
);
pm.sendRequest(request, (err, res) => {
  const rows = res.json().data || [];
  // ... console.log de diagnostico
});
```

Para un caso **negativo** (sección 5), la única diferencia es agregar **una sola línea** antes del chequeo de BD:

```
// Sobreescribe SOLO 'monto' -- las demas variables quedan en su default.
pm.collectionVariables.set("monto", -500);
```

Por qué es mejor que clonar un objeto: no hay `JSON.parse` / `JSON.stringify` , no hay que reconstruir el body a mano, y cualquiera que lea el Pre-request Script ve inmediatamente *qué campo cambia* para ese caso — una línea, un campo, sin ruido alrededor.

3.4 — Tests Script (post-response): la segunda validación

```
pm.test("Status 201 Created", () => { pm.response.to.have.status(201); });

const body = pm.response.json();
const transferenciaId = body.data.id;

// Post-request: vuelve a consultar la BD (mismo helper utils.bodySqlRest)
// para confirmar que el INSERT realmente quedo persistido.
const request = utils.bodySqlRest(
  "SELECT id, cuenta_origen_id, cuenta_destino_id, estado FROM transferencias WHERE id = $1",
  [transferenciaId]
);
```

```
pm.sendRequest(request, (err, res) => {
  pm.test("La BD confirma la fila insertada con estado 'pendiente'", () => {
    const row = res.json().data[0];
    pm.expect(row).to.not.be.undefined;
    pm.expect(row.estado).to.eql("pendiente");
  });
});
```

- A** Primer `pm.test`: valida la respuesta HTTP (lo que cualquier test "normal" haría).
- B** Se extrae el `id` que la API devolvió — es el dato que necesitamos para ir a buscar esa fila específica en la base.
- C** Segundo `pm.sendRequest` + `pm.test` anidado: este es el que realmente prueba el efecto en la BD. Nota que `pm.test` puede llamarse dentro del callback de `pm.sendRequest` — Postman lo sigue contando como un test más en los resultados, aunque sea asíncrono.

4 Diagrama completo del flujo (caso feliz)



5 Reusar el patrón para un caso negativo

Con los defaults ya declarados en el pre-request de la colección, generar un caso de borde es sobrescribir **una sola variable** en el pre-request del request y ajustar la aserción del test:

Caso	Sobrescritura en el pre-request	Qué se espera	Cómo se prueba en la BD
Monto inválido	<code>pm.collectionVariables.set("monto", -500);</code>	Zod (<code>monto: positive()</code>) rechaza con 400 antes de tocar la BD	<code>COUNT(*)</code> de <code>transferencias</code> antes vs. después: debe ser igual (no se insertó nada)
Cuenta origen inexistente	<code>pm.collectionVariables.set("cuentaOrigenId", 999999);</code>	Pasa Zod, rompe la <i>foreign key</i> al insertar (Postgres <code>23503</code>) → 400	Mismo <code>COUNT(*)</code> antes/después

```
// Pre-request del caso "monto invalido". Unica linea distinta del caso feliz:
// cuentaOrigenId/cuentaDestinoId/descripcion quedan en su default.
pm.collectionVariables.set("monto", -500);

const request = utils.bodySqlRest("SELECT COUNT(*) AS total FROM transferencias");
pm.sendRequest(request, (err, res) => {
  pm.variables.set("totalAntes", res.json().data[0].total); // pm.variables: solo
}); // para este request

// Test del mismo request
pm.test("Status 400 Bad Request (monto invalido)", () => {
  pm.response.to.have.status(400);
});
const request2 = utils.bodySqlRest("SELECT COUNT(*) AS total FROM transferencias");
pm.sendRequest(request2, (err, res) => {
  pm.test("La BD no registro ninguna fila nueva", () => {
    pm.expect(Number(res.json().data[0].total)).to.eql(Number(pm.variables.get("totalAntes")));
  });
});
```

Nota didáctica: acá se usa `pm.variables` (no `pm.collectionVariables`) para `totalAntes`, porque ese dato solo hace falta *dentro de este mismo request* — no necesita viajar en el body ni ser visto por otros items. Guardar en collection variables solo lo que realmente necesita ser global evita que la colección se llene de variables "basura".

6 Los 5 requests de la carpeta E2E - Flujos con validación SQL

Request	Operación	Pre-request valida	Test (post-response) valida
Crear transferencia	INSERT	Cuentas 1 y 2 activas	201 + fila en <code>transferencias</code> con <code>estado='pendiente'</code>
Crear transferencia – monto inválido	INSERT rechazado	Cuenta el total de filas antes	400 + el total no cambió
Crear transferencia – cuenta origen inexistente	INSERT rechazado (FK)	Cuenta el total de filas antes	400 + el total no cambió
Pagar factura	UPDATE transaccional	Factura 3 aún no está <code>'pagada'</code>	200 + <code>facturas.estado='pagada'</code> y fila nueva en <code>pagos</code>
Revocar rol de usuario	Soft-delete (UPDATE)	El rol está activo para ese usuario	204 + <code>usuario_rols.activo=false</code> (la fila sigue existiendo, no se borra)

7 Cómo aplicar el patrón a otro endpoint

- 1 Identificá qué tabla(s) toca el endpoint (ver la tabla de `qa_training` en el `README.md`) y qué columna refleja el resultado de la operación (un `estado`, un `activo`, etc.).

- 2 En el Pre-request Script del request, escribí un `SELECT` con `utils.bodySqlRest(...)` que confirme la precondition (¿existe el dato? ¿está en el estado esperado antes de mutarlo?).
- 3 Si el endpoint tiene un body con varios campos y querés probar casos de borde, declará una collection variable con su default por cada campo que en algún caso vaya a variar (en el Pre-request Script de la **colección**, una vez) y usá `{{esaVariable}}` en el Body — igual que en la sección 3.2. Cada caso negativo solo sobrescribe la variable puntual que necesita, como en la sección 5.
- 4 En el Tests Script, primero afirmá el status code y la forma de la respuesta; después, con el id/clave que devolvió la API, hacé un segundo `utils.bodySqlRest(...)` que confirme el estado final en la tabla.
- 5 Para casos negativos, en vez de leer una fila puntual (todavía no existe), comparar un `COUNT(*)` antes/después es la forma más simple de probar "no pasó nada en la BD".

8 Glosario rápido de `pm.*` usadas

Método	Para qué
<code>pm.collectionVariables.get/set(key, value)</code>	Leer/escribir una variable de colección (persiste entre requests).
<code>pm.variables.get/set(key, value)</code>	Leer/escribir una variable local (solo vive durante ese request).
<code>pm.sendRequest(options, callback)</code>	Manda un HTTP request extra desde un script; el callback recibe <code>(err, response)</code> .
<code>pm.response.json()</code>	Parsea el body de la respuesta como JSON.
<code>pm.response.to.have.status(n)</code>	Asercion de status code.
<code>pm.test(nombre, fn)</code>	Declara un test con nombre; aparece en los resultados del Runner/Newman.
<code>pm.expect(valor).to...</code>	Aserciones estilo Chai (<code>.eq</code> , <code>.to.not.be.undefined</code> , etc.).

Este documento acompaña la carpeta `E2E - Flujos con validación SQL` de `postman_collection.json` en el repositorio del curso. Cualquier duda sobre una línea puntual de los scripts, comparala contra el request real en Postman — los comentarios `//` dentro de cada script siguen la misma numeración de pasos que esta guía.